

Data Structure Using C++

Lecture : 3

(Linked Lists)

Dr. Ali Hamzah



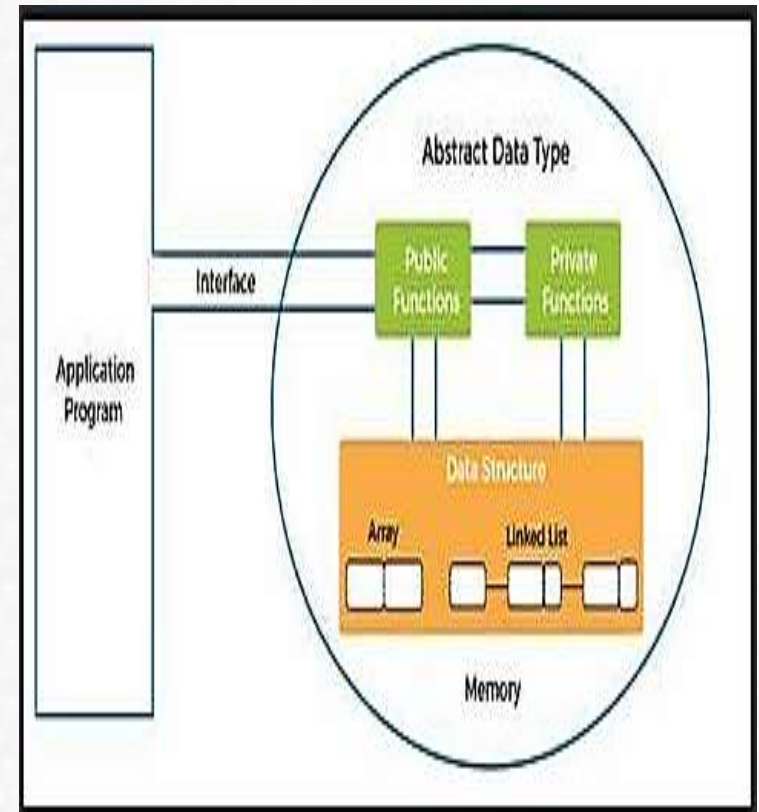
Review *Abstract Data Types (ADT)*



Abstract data type (ATD)

■ For what purpose?

- To facilitate efficient
 - » Storage of data
 - » Manipulate of data
 - » Retrieval of data



■ **Collection of data in an ordered fashion**

- Ages of the students in a class are all numbers (data), but once are grouped in one line, one after the other becomes **structured data** (arrays)
- Age, class, sex, height, weight of a student is all data, but if we place them in one line per student it would be structured data (structures/struct)
- Arrays, list, queues, stack, doubly linked lists, structures etc are all data structures

■ **Different operations** are defined for each type of data structure to add, delete or modify its content

Review *Data Structures ADT*

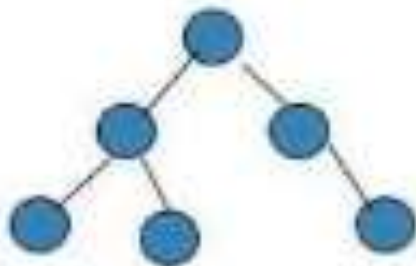
Types of data structures



Array



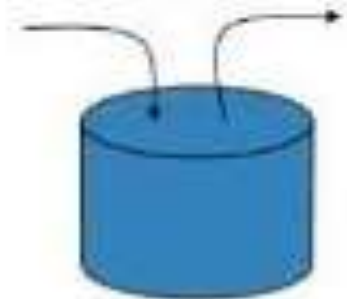
Linked List



Tree



Queue



Stack

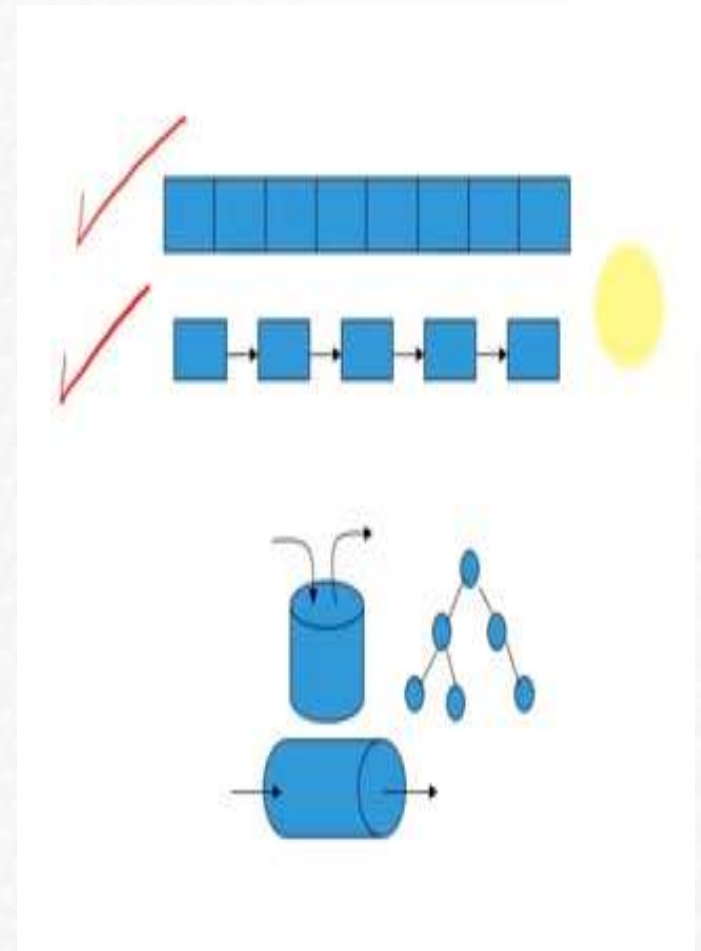
➤ Based Existence

❖ Physical data structures

- Array
- Linked List

❖ Logical data structures

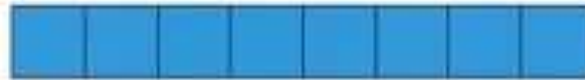
- Can't be created independently
- Stack, queues, trees, graph



➤ Based on Memory Allocation

❖ Static(or fixed sized) data structure

- Such as Array



❖ **Dynamic data structure (change size as needed)**

- Such as linked list



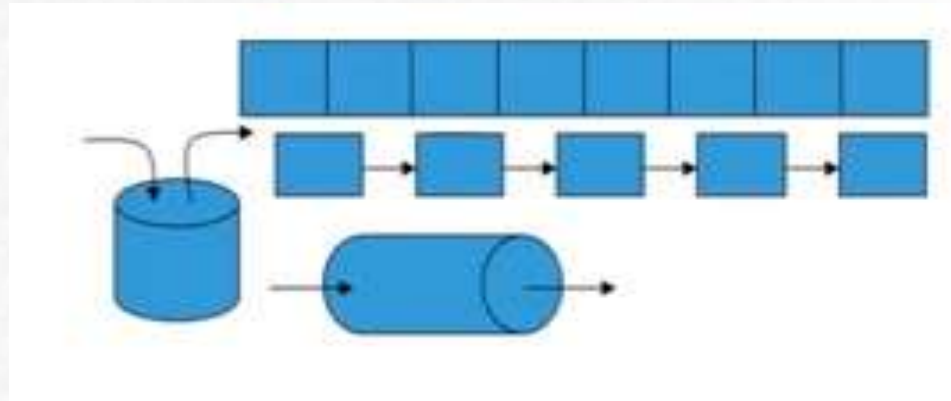
Review *Classification of data structure*



➤ Based on Representation

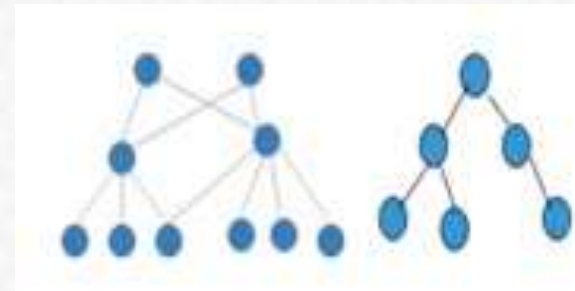
❖ Linear Data structures

- Array
- Linked list
- Stack
- queue



❖ Non Linear Data structure

- Trees
- Graph



Review *Operation of data structure*



■ Traversing

- Accessing/visiting each data element exactly once so that certain elements in the data may be processed

■ Searching

- Finding the location of the given data element(key)

■ Insertion

- Adding a new element to the structure

Review *Operation of data structure*



■ **Deletion**

- Removing element data from the structure

■ **Sorting**

- Arrange the data elements in some logical fashion
 - **Ascending/descending**

■ **Merging**

- Combine data elements from two or more data structure into one

- The operations defined on the data structures are generally known as algorithms
- For each data structure there has to be defined algorithms
- Algorithms are normally used to add, delete, modify, sort items of a data structure
- All programming languages are generally equipped with conventional data structures and algorithms.
- New data structures and the algorithms for manipulation can be defined by the user

■ Design Issue

» *The challenge is to select the most appropriate data structure for the problem*

Second Type :

Linked List

What is wrong with arrays?



- ++ Fast access of data with indices
- Size has to be known at compile time
- Resizing needs re-location
- Insertion and removal
 - Data are continuous in memory
 - Data need to be shifted left or right

Single Linked Lists

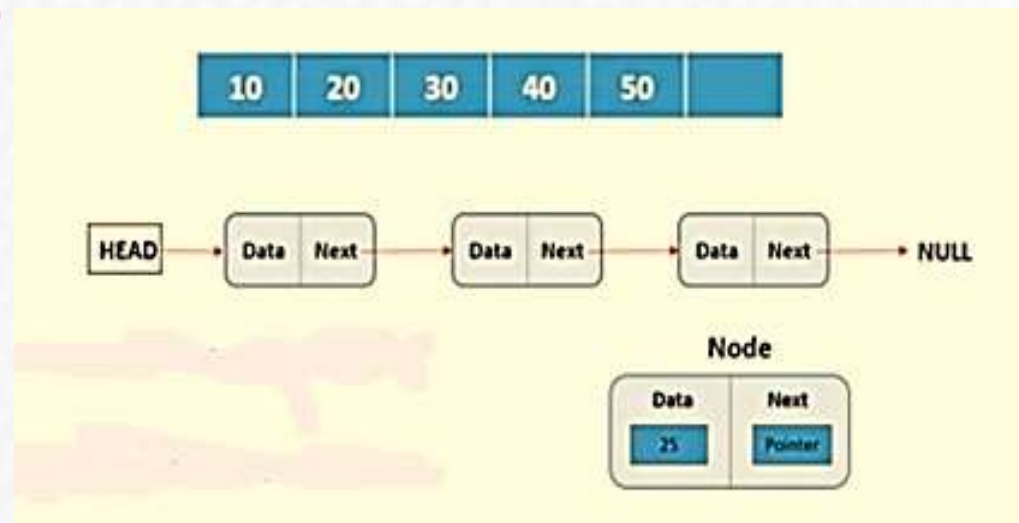


- **Linked Lists** avoid these problems by storing data in nodes that are linked by pointers.
- *Main operations:* insert, delete, search and others
- *Better performance* **than** arrays in some operations and **worse** performance in others.

Single Linked Lists



- **Why Linked Lists ?**
- ✓ **Array can be used to store elements linear data of similar type.**
- **But arrays have the following limitations:**
 - the size of array is fixed
 - inserting a new element in array is expensive
 - deletion is also expensive unless special technique is used
- ❖ **Advantage over arrays**
- ✓ **Dynamic Size**
- ✓ **Ease of insertion / deletion**



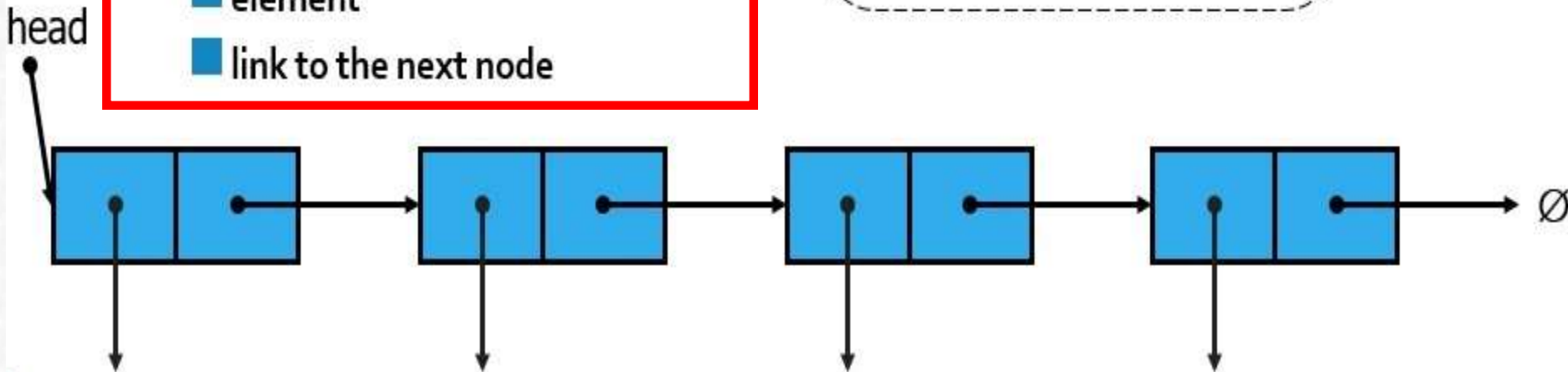
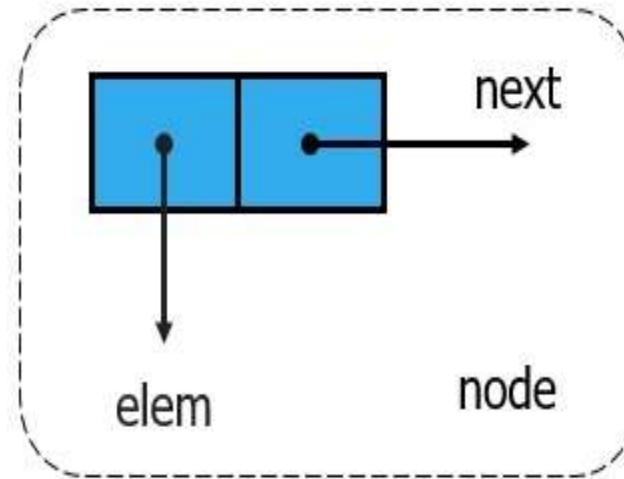
Single Linked List

◆ A singly linked list is a concrete data structure consisting of a sequence of nodes, starting from a head pointer

◆ Each node stores

■ element

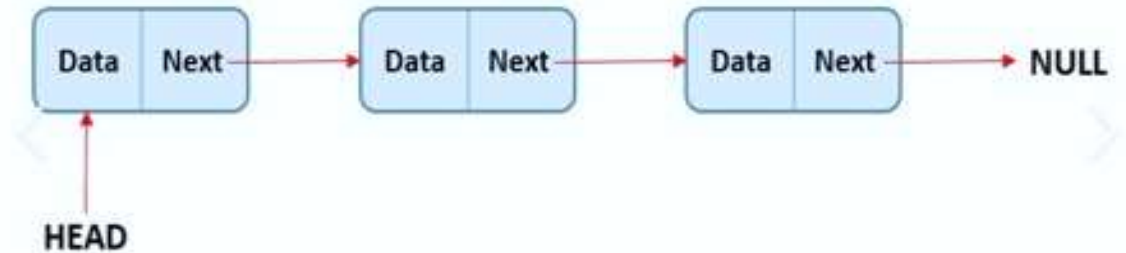
■ link to the next node



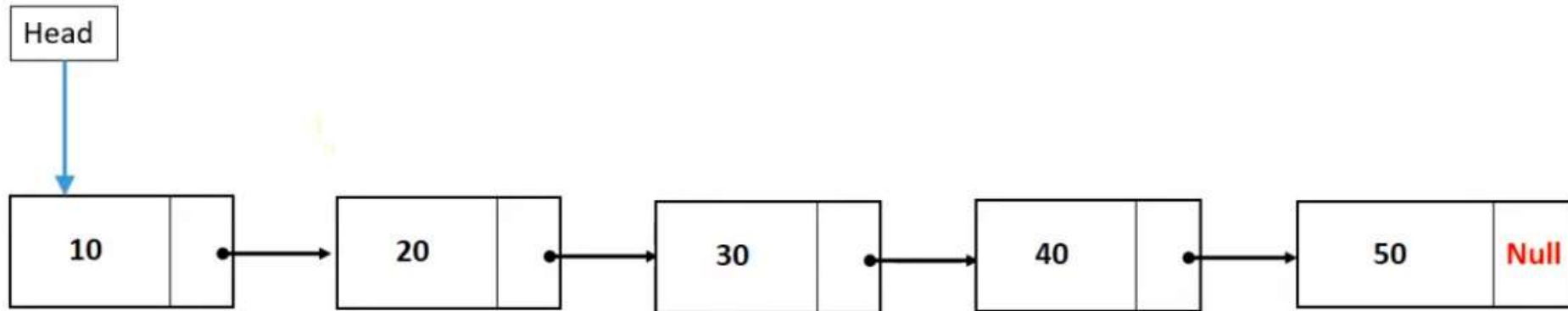
Linked List Operation



- Create ()
- Traverse / Display ()
- Count()
- Search ()
- Insert (First , Last , before specific Item)
- Delete (First , Last , Specific Item)
- Replace ()



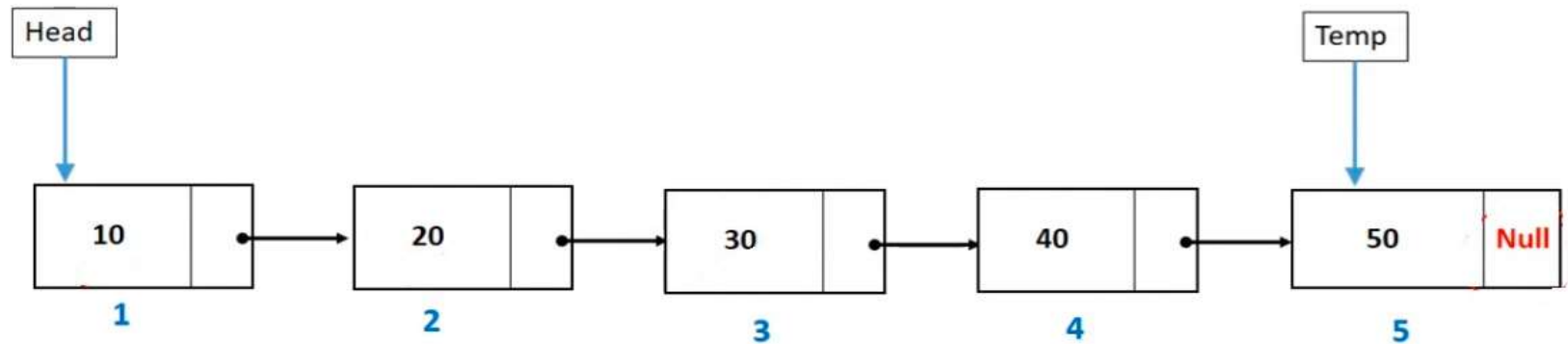
Traverse



Head = Head->next

```
node *temp = head;  
Cout <<("List elements are - \n");  
while(Team->next != NULL) {  
    Cout <<(" --->",temp->data);  
    temp = temp->next;  
}
```

Traverse



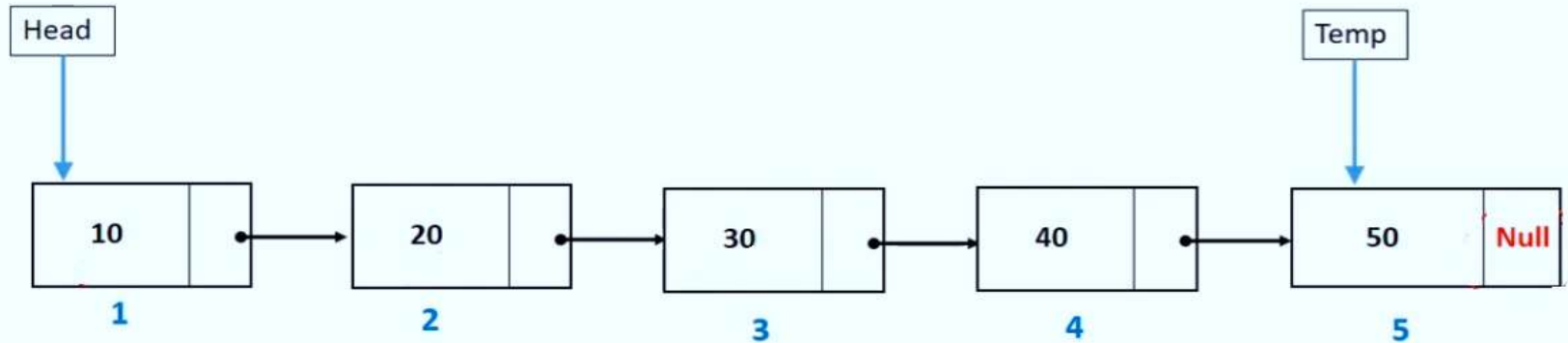
`Node* temp = Head;`

`Temp = Temp->next`

Temp = Null

Temp	Printed Output
1	10
2	20
3	30
4	40
5	50

Searching



Node* temp = Head;

Temp = Temp->next

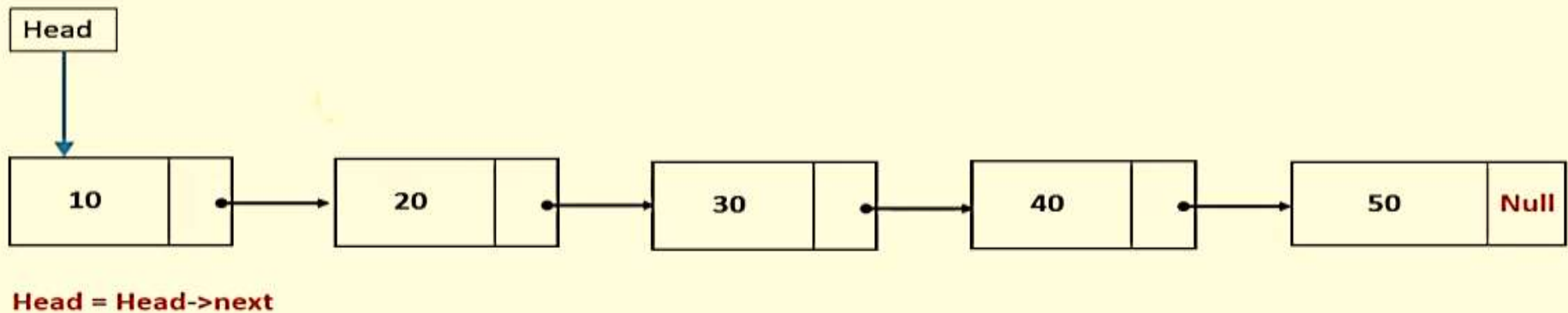
Temp = Null

Temp	Printed Output
1	10
2	20
3	30
4	40
5	50

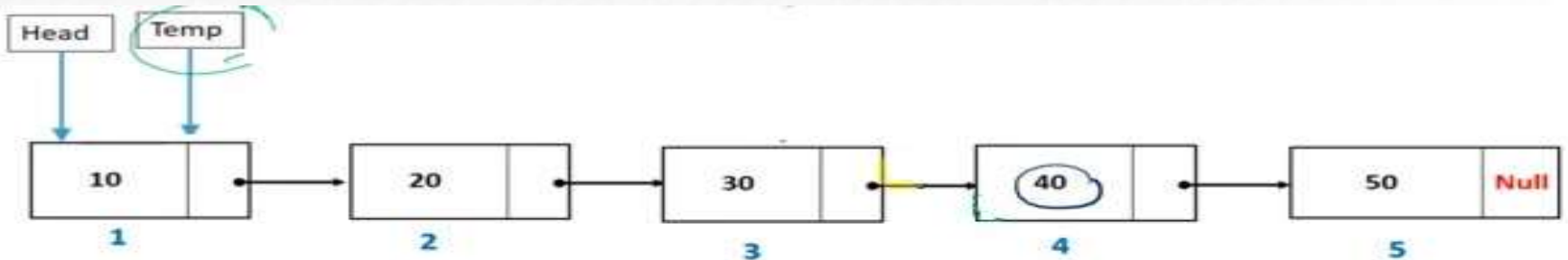
Insertion in linked list



- ❖ There are various positions where node can be inserted
 - Insert at front(as a first position)
 - Insert at middle (any position)
 - Insert at end (as a last node)



Insert at front(as a first position)



```
Node *temp = Head;  
Node *newnode  
Head = newnode  
Newnode->next =temp
```



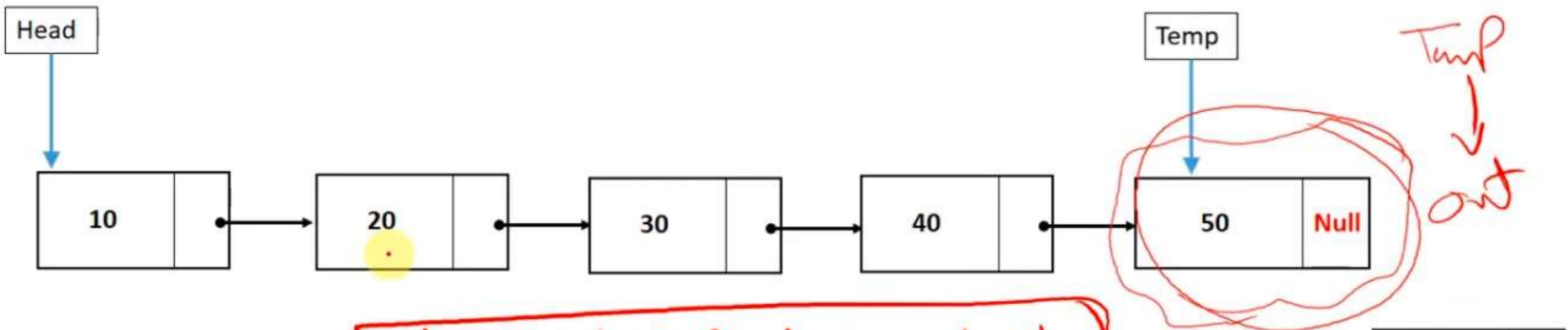
InsertBefore (40, 35);

```
Temp->next = newnode;
```

Append(item)



Append (60);



```
Node* temp = Head;
```

```
Temp = Temp->next
```

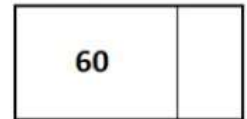
```
Node *newnod ;
```

```
newnode->data = 60 ;
```

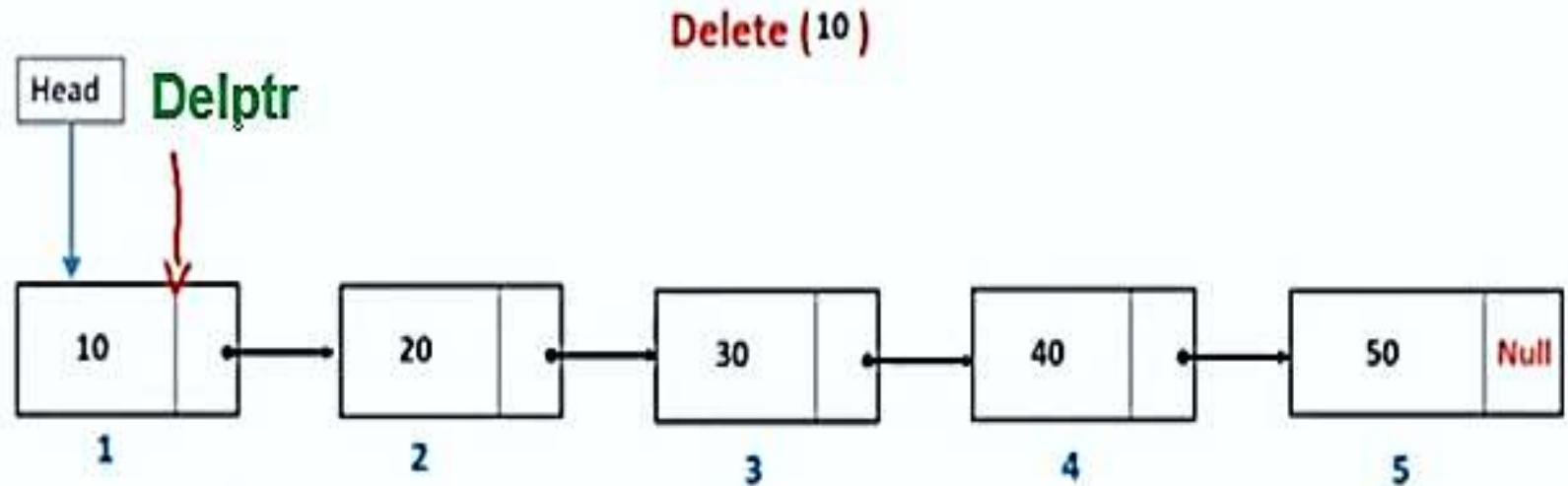
```
temp->next = newnode;
```

```
newnode->next = NULL;
```

~~while (Temp != Null)~~
while (Temp->next != null)



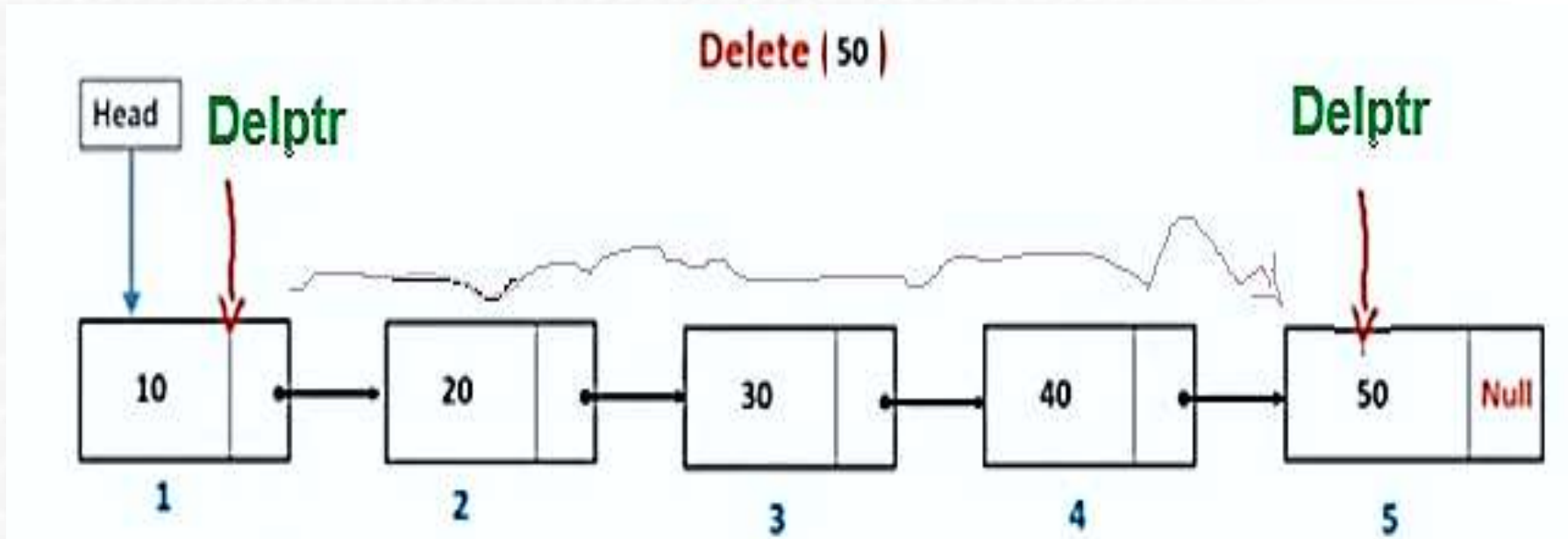
Delete (First Element)



Delptr = Head
Head = Head → next;
Delete Delptr;



Delete (Last Element)



```
Delptr = Head  
Delptr = Delptr → next;  
Delete Delptr;
```

Linked List



ARRAY

1. Arrays are stored in contiguous location.
2. Fixed in size.
3. Memory is allocated at compile time.
4. Uses less memory than linked lists.
5. Elements can be accessed easily.
6. Insertion and deletion operation takes time.

LINKED LISTS

1. Linked lists are not stored in contiguous location.
2. Dynamic in size.
3. Memory is allocated at run time.
4. Uses more memory because it stores both data and the address of next node.
5. Element accessing requires the traversal of whole linked list.
6. Insertion and deletion operation is faster.

Doubly Linked Lists

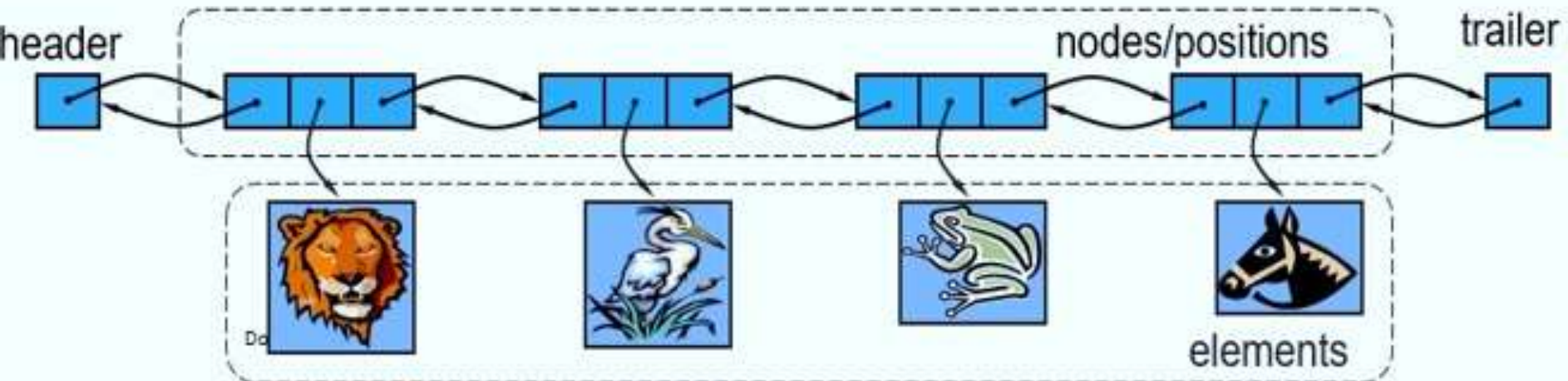
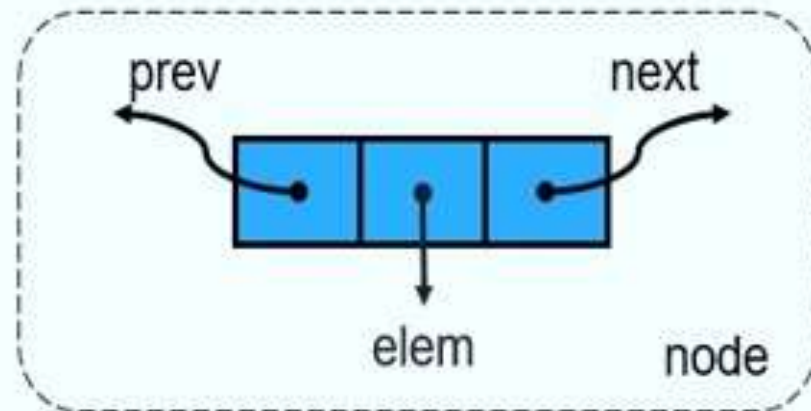
Doubly Linked Lists



- Traversed in either direction
- Typical operations
 - Initialize the list
 - Destroy the list
 - Determine if list empty
 - Search list for a given item
 - Insert an item
 - Delete an item, and so on

Doubly Linked Lists

- A doubly linked list provides a natural implementation of the Node List ADT
- Nodes implement Position and store:
 - element
 - link to the previous node
 - link to the next node
- Special trailer and header nodes



Doubly Linked Lists (cont'd.)



- Linked list in which every node has a `next` pointer and a `back` pointer
 - Every node contains address of next node
 - Except last node
 - Every node contains address of previous node
 - Except the first node

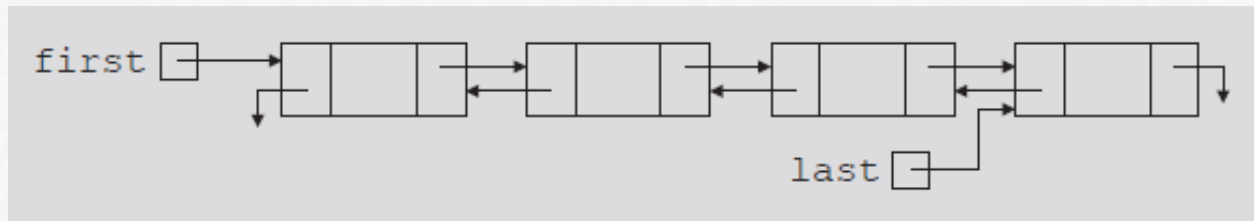


FIGURE 5-27 Doubly linked list

Doubly Linked Lists (cont'd.)



```
struct Node {  
    int data;  
    struct Node* next;  
    struct Node* prev;  
};
```

